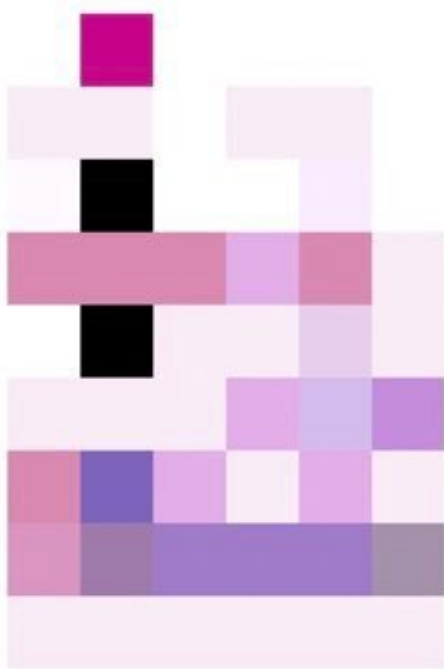


3>SHAKE

AIで実装は速くなった。
なのにプロダクトは速くならない。
職能の壁を越えて、価値が届くまでの流れを設計する

2026/09/05 Product Engineering Conference 2026
@nwiizo 30min



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。格闘技、読書、グラビアが趣味でよく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていく。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



スリーク

日本のSREをリードする

SRE総合支援からセキュリティ対策を全方位支援

Data Engineering



レコナー

あらゆるサービスを連携するハブになる

クラウド型ETL/データパイプラインサービスの決定版

Security



セキュリファイ

事業者が抱えるセキュリティリスクを無くす

ワンストップで実現するセキュリティ対策

HR (Engineer Hiring)



リランス

良いエンジニアに良い案件を

フリーランスエンジニアに「今よりいい条件」を



初めての単著が出ました



書影：ダイヤモンド社

『おい、とりあえず終わらせろ』

完璧に準備しようとして動けなくなるより、まず終わらせ、そこから直していく。そんな進め方をまとめた本です。

2026年に翻訳に携わった本

『アーキテクチャモダナイゼーション』

『セキュアAPI』

『実践 プラットフォームエンジニアリング』

最近では本を読むだけでは飽き足らず、書いたり、訳したりしています。

『アーキテクチャモダナイゼーション』の翻訳で考えてきた「価値が届くまでの流れ」も、今回の発表につながっています。

この発表で解決できること

こんな状況ではありませんか

AIでPull Request（PR）は増え、手を動かす速さも上がったように感じる。けれど、価値が届くまでの時間や成果は、同じようには変わっていない。**速くなった実感と、届いた価値が噛み合わない。**

持ち帰れるもの

なぜ、実装が速くなってもプロダクト全体は速くならないのか。価値が届くまでの流れを測り、**全体の速さを決める詰まり**を見つけます。ユーザーが達成したいこと、失敗のを見つけやすさ、利用状況とほかへの影響から、「**作る・任せる・やめる**」を決める**基準**も持ち帰れます。

見る範囲を、実装からユーザーの結果まで広げる。すると、次に何を速くすべきかが見えてくる。

先に白状します。伝えたいことを絞り切れず、スライドが多くなりました。今日は何枚か飛ばします。すべてを読もうとせず、**気になった言葉と、話がどうつながるか**だけ追ってください。細部は公開版で、あとからじっくり読めます。



本日の流れ

1. 増えた生産量はどこに消えたのか
2. 測りやすい数字は、価値が届いた証拠ではない
3. 役割ごとの前提を対話で確かめる
4. ユーザーが達成したいことから「作る・任せる・やめる」を判断する

まず、ユーザーにとっての価値を確認し、実装を速めても全体が同じだけ縮まない理由を数字で見ます。作業と待ちを測ったうえで、利用後に何が変わったかを確認めます。次に、役割ごとの前提を対話と図で具体化します。最後に、ジョブから「作る」を決め、確かめられる範囲をAIへ「任せる」。既存の動作と依存を調べ、「やめる」条件や置き換え方まで考えます。



1. 増えた生産量はどこに消えたのか

個人は速くなったと感じる。組織の流れは別に測る。

AIで速くなった実感を、結果と分けて確かめる

AIでコードを書きやすくなったと感じても、確認や修正を含む作業時間まで短くなったかは、別に確かめる必要があります。さらに、**個人の作業時間**と、**ユーザーや事業に起きた変化**も分けて測ります。

個人は速くなったと感じる

DORAは、ソフトウェア開発組織を継続的に調査しています。2025年の調査では、世界の技術職約5,000人のうち90%が仕事でAIを使い、80%以上が生産性の向上を感じている。

個人の体感だけでは組織の流れは決まらない

同じ調査では、AIをよく使う組織ほど、一定期間にユーザーへ届けた変更の件数が多く、ユーザーや事業の成果も高い傾向がありました。その一方で、変更による障害や手戻りは増えていました。DORAの2026年レポートも、コーディングが速くなるだけでは投資対効果（ROI）につながらないとしています。

AIを入れると、組織の強みは伸び、弱いところの問題も大きくなる。個人の体感と、ユーザーが結果を得るまでの流れは、別の指標で見る。



価値の基準を、ユーザーの目的に置く



クレイトン・M・クリステンセンほか『ジョブ理論』

実装の速さをユーザー側の結果へつなげるには、まず「何を達成したいのか」を考えます。ユーザーがなぜ製品やサービスを選ぶのかを、達成したいことから考える方法が**Jobs to Be Done**（ジョブ理論）です。ここでいうジョブは、ユーザーがある状況で片付けたいことです。

状況 候補が多く、比較に時間がかかっている

ジョブ 必要な情報へ早くたどり着き、購入を判断したい

選べる手段 検索条件、比較表、担当者への相談

ジョブ理論では、ユーザーはジョブを片付けるためにプロダクトを「雇う」と表現します。検索機能は手段の一つです。別の手段の方がうまく片付くなら、ユーザーはそちらを選びます。

ジョブは機能名ではない。ユーザーが置かれた状況と、達成したいことを表す。

出典：Christensen Institute, Jobs to Be Done Theory / 書影：クレイトン・M・クリステンセンほか『ジョブ理論』

完全に余談なのですが

ここは発表では飛ばします。公開版だけの余談です。

クレイトン・M・クリステンセンの本では、『ジョブ理論』以外に、この二冊も好きです。

『イノベーションのジレンマ』

優良企業は、既存顧客の要望を聞き、収益の上がる改善へ投資します。その合理的な判断が、当初は主流顧客の求める性能に届かず、市場も小さい新技術への対応を遅らせます。

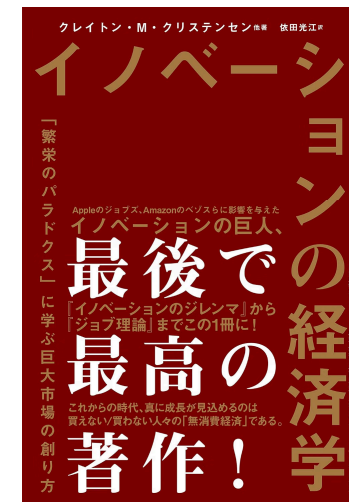
新技術が別の顧客に受け入れられ、改良を重ねて既存製品を脅かす。この動きを**破壊的イノベーション**として説明した本です。

『イノベーションの経済学』

「繁栄のパラドクス」に学ぶ巨大市場の創り方

高価、複雑、手に入りにくいといった理由で、既存の製品を利用できない人々がいます。本書では、この状態を**無消費**と呼びます。

手頃で使いやすい解決策が新しい市場をつくり、販売や流通、雇用も育てていく。これを**市場創造型イノベーション**として、各国の事例から説明します。



クレイトン・M・クリステンセン
ほか『イノベーションの経済学』

いま見えている顧客だけを前提にすると、次に生まれる市場を見落とす。この視点が好きです。

出典：翔泳社『イノベーションのジレンマ 増補改訂版』 / ハーパーコリンズ・ジャパン『イノベーションの経済学』

この発表では、商品を選ぶ場面を例に考える

ここから繰り返し使うのは、仕事で使う商品を購入する架空のサービスです。利用者は商品名だけでなく、必要な日までに届くか、条件に合うかを確認して、購入する商品を決めたいと考えています。

利用者が困っていること

候補が多く、商品ごとに納期や条件を調べ直している。
検索結果が出ても、どれを買えばよいか決められない。

チームが考えている変更

納期で絞る機能、比較しやすい表示、検索の応答時間の改善。ただし、どれが困りごとに効くかは、まだ確かめる必要がある。

以降の「検索」は、この利用場面を指します。検索画面の使いやすさ、裏側の処理速度、情報の正確さを、同じ「速くする」で片付けずに考えます。個別の経験談は、この架空例と分けて紹介します。

目指すのは検索機能の完成ではなく、利用者が条件に合う商品を選ぶこと。



実装が終わっても、価値が届いたかはまだ分からない

実装完了は、予定した機能がコードとして動き、テストを通った状態です。リリースすれば、ユーザーが利用できるようになります。ここまでで確認できるのは、**チームが予定したものを作り、利用できるようにしたこと**です。

必要な人が利用できるか

対象のユーザーが機能を知り、必要なときに使えるとは限りません。

実際の状況で使われるか

使える機能でも、ユーザーが別の手段を選ぶことがあります。

望んだ結果が起きるか

使われても、探す時間や購入の判断が変わらないことがあります。

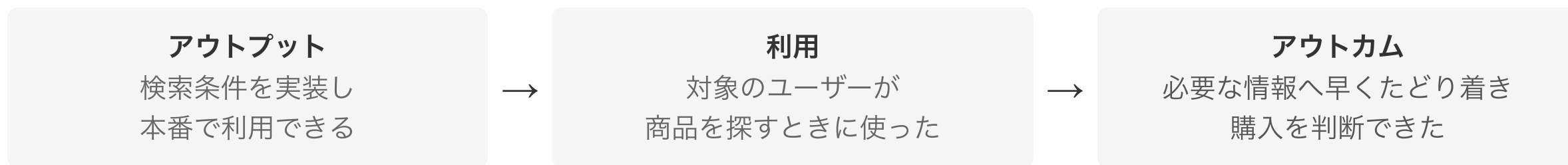
価値が届くのは、実装が終わったときではなく、ユーザーのジョブが片付いたときです。



価値が届くのは、ユーザーのジョブが片付いたとき

ジョブが片付いたとは、機能を使った結果、ユーザーが望んでいた変化が実際に起きた状態です。検索機能を実装できても、必要な情報へ早くたどり着けなければ、ジョブはまだ片付いていません。

作って出したものを**アウトプット**、利用後に生じたユーザーや事業の変化を**アウトカム**と呼びます。



利用回数はアウトプットとアウトカムをつなぐ手がかりです。ただし、利用回数だけではジョブが片付いたか分かりません。利用後の変化まで確かめます。

プロダクトの速さは、機能を出すまでではなく、結果を確かめるまでで見る必要があります。

たとえば、実作業が30%の流れなら

まず、アウトプットを本番で利用できる状態にするまで、つまりチケットに着手してから本番へ反映するまでを、実作業30%、待ち70%と置いた例で考えます。

実作業 30%

設計 ・ 実装 ・ テスト

待ち 70%

判断待ち ・ レビュー待ち ・ 他チーム待ち

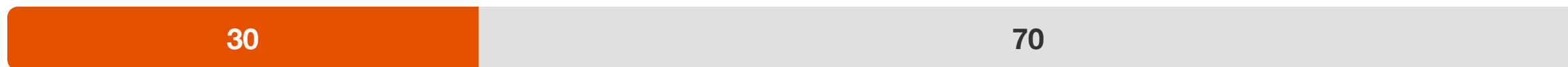
この例では、実作業以外の7割が、仕様や範囲が決まるのを待つ時間、レビューと承認を待つ時間、依存先のチームの変更やリリースを待つ時間です。

比率は説明のための仮定です。自分たちの比率は、実際の記録から測ります。



実作業を半分にしても、全体は15%しか縮まない

いま



AIで実作業を半分にした後



全体を100と置くと、待ちの70は、実作業だけをAIで速めても残ります。実作業を半分にしても、全体の短縮は15%にとどまります。

待ちが変わらないという仮定なら、残る時間の多くは待ちになる。
次に、どの仕事で時間がかかっているかを分けて見る。

実作業を速めても、外側の待ちは残る

先ほどの30/70の例を、AIを使う場面に重ねます。AIが直接短くしやすいのは、一人が手元で作って確かめる繰り返しです。これを**内側のループ**と呼びます。その成果をチームが受け入れ、利用できる状態にする繰り返しが**外側のループ**です。

内側のループ

設計し、コードを書き、動かし、手元でテストする。実作業30%と重なる部分が多く、AIで短くしやすい。

外側のループ

優先順位を決め、レビューし、統合し、リリースする。役割をまたぐ判断や調整があり、待ち70%が生まれやすい。

AIで内側の成果が早く出ても、外側で一日に受け入れられる件数は自動では増えません。レビューの観点、判断する人、依存先との進め方を変えなければ、早くできたPRが外側の工程に並びます。

内側・外側は仕事の範囲、実作業・待ちは時間の使われ方。
レビューにも実作業はあり、AIで短くできる部分と、判断待ちが残る部分を分けて測る。



バリューストリームは発見から結果の確認まで

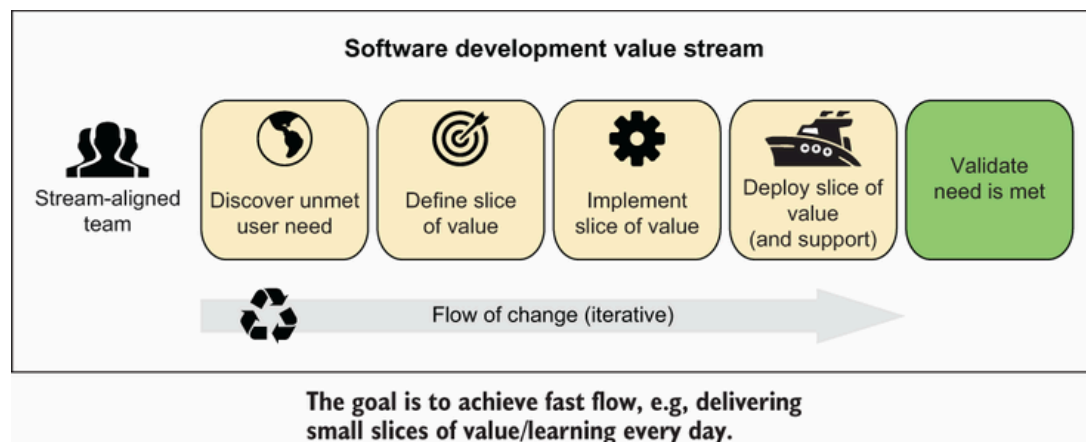


Figure 6.1 The high-level activities in an independent value stream より引用

内側と外側のループで整理したのは、主にチケットへ着手してからアウトプットを利用できる状態にするまでです。しかし、プロダクトの仕事は着手前から始まり、リリース後も続きます。ユーザーのジョブを見つけ、求めるアウトカムと確かめ方を決め、アウトプットを届け、ジョブが片付いたか確かめるまで。この全体をバリューストリームと呼びます。

見る範囲の始まり ユーザーの困りごとを捉えたとき
一回の検証の区切り 結果を確かめ、続けるか見直すか決めたとき

出典：Susanne Kaiser, *Architecture for Flow*, Addison-Wesley, 2025

実装は、バリューストリームの真ん中にある

範囲をバリューストリームまで広げると、実装の位置づけが見えてきます。実装は、その途中でアウトプットを作って届ける仕事です。前には、ジョブを見つけて何を作るかを決める仕事があり、後には、利用後のアウトカムを確かめる仕事があります。

ジョブを見つける

ユーザーが困る状況、いま試している手段、達成したいことを、観察や対話から確かめる。

アウトカムと確かめ方を決める

ユーザーの行動や事業の数字がどう変われば価値が届いたと言えるかを、実装前に決める。

アウトプットを作って届ける

設計し、実装してテストする。リリースして、ユーザーが変更を利用できる状態にする。

アウトカムを確かめる

実際に使われ、期待した変化が起きたかを見る。続ける、変える、やめるを判断する。

つまり、アウトプットを作る前には判断があり、届けた後にはアウトカムの確認があります。実装だけを速くしても、前後が遅ければ、バリューストリーム全体は速くなりません。



バリューストリームマッピングは、作業と待ちを分ける

バリューストリームの工程を時系列に並べ、各区間を実際に手を動かしていた時間と、判断や作業を待っていた時間に分けて可視化する方法を、**バリューストリームマッピング**と呼びます。

直近の案件を並べる

まず10件程度。
未完了の案件も含める

→

時刻を記録から拾う

ジョブの確認、着手、PR、承認、
本番反映、利用開始、結果確認

→

区間を分ける

作業か、誰かを待ったか。
待った相手の役割を書く

実際の記録を使い、作業していた時間と待っていた時間を分ける。

一件の変更を、同じ時間の単位でたどる

測り方も、検索条件を追加する架空の例で考えます。着手から本番反映までを、夜間や休日を除くチームの共通の業務時間で測り、合計20時間だったとします。

実際に進めた時間：6時間

設計・実装・テストに5時間。レビューで内容を確認した時間が1時間。

止まっていた時間：14時間

レビュー開始を待って10時間。本番へ反映する判断を待って4時間。

この例なら、実作業の割合は $6 \div 20$ で30%です。二人で同時に1時間確認しても、案件が進んだ経過時間は1時間です。人数分の工数を足すと、全体に占める割合とは別の数字になります。

時刻の差だけでは、作業と待ちは分らない。
記録と担当者への確認を組み合わせ、不明な時間は不明として残す。

待ち時間は、相手と理由まで記録する

工程を並べたら、最初に二つを見ます。個人の働き方を評価するためではありません。役割の間で仕事をどう渡し、どこで止まっているかを知るためです。

作業していた時間の割合

開始と終了を揃えた期間のうち、設計、実装、確認など、案件を実際に進めていた時間がどれだけあったか。

最も長く待った工程

どの工程で、誰の、どんな判断や作業を待ったか。件数だけでなく、待った理由も記録する。

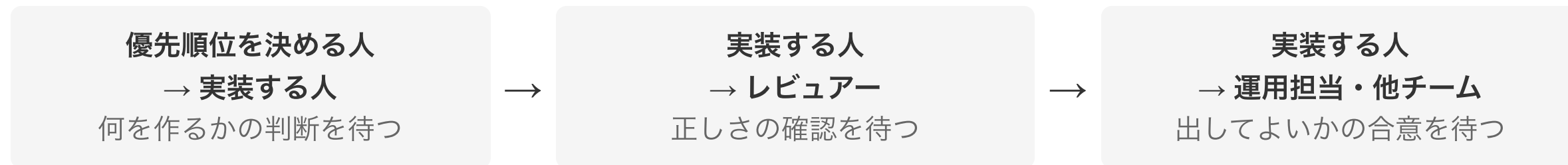
待ちのすべてが無駄ではありません。本番前の確認のように、安全のために設ける待ちもあります。担当と判断日が決まった保留は、期限のない放置と分けます。見直すのは、目的を説明できない確認、担当が曖昧な引き継ぎ、処理できる量を超えて積み上がる待ちです。

待ちが大きい工程を、全体の速さを決めている候補として詳しく見る。相手と理由が分かれば、減らす方法を選べる。



仕事を引き継ぐところで、待ちが生まれやすい

待ちの相手と理由が分かったら、仕事を渡す境目を見ます。ここでは、変更を利用できるようにするまでに、誰から誰へ仕事を渡し、何を待っているかを確かめます。



ここでは、判断や作業を一つの役割から別の役割へ引き継ぐところを**責任の境界**と呼びます。この例では、別の役割の判断が必要になるたびに待ちが発生していました。引き継ぐたびに説明と調整が加わり、次の作業へすぐには進めなくなります。

AIで作業を速くしても、引き継ぐ回数と待つ順番は変わらない。

依存は、待つ理由で三つに分ける

依存とは、自分たちだけでは次へ進めない状態です。何を待っているかで、対処が変わります。

仕組みへの依存

別のシステムや接続先の変更を待つ
→ 接点を安定させ、別々に変更できるようにする

人の知識への依存

特定の人に聞くまで判断できない
→ 判断の根拠と手順を共有する

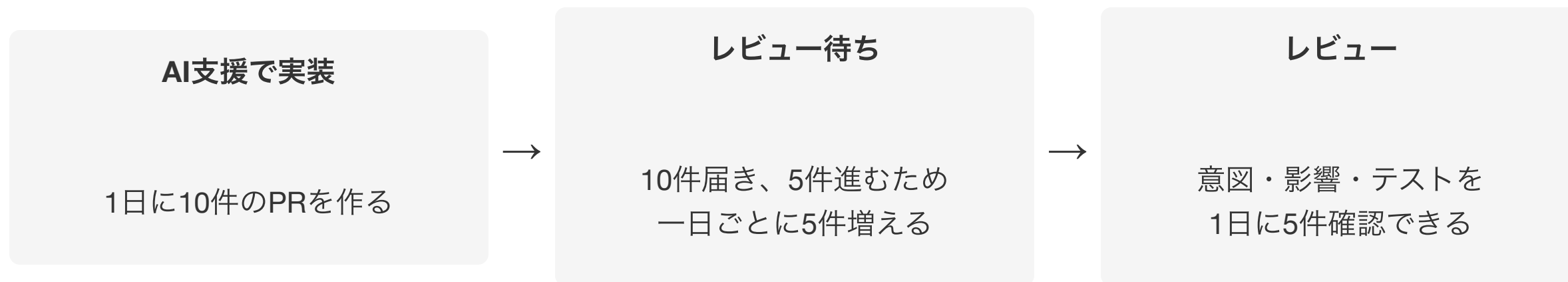
順番への依存

先の確認や承認が終わるまで進めない
→ 同時に進めるか、確認条件を明示する

依存をゼロにはしない。待ちが長い依存から、本当に必要かを確認、待ちを減らす方法を決める。

全体の速さを決める工程から直す

各工程には、一日に処理できる仕事の量があります。実装から渡される量が次の工程で処理できる量を超えると、終わっていない仕事はその手前に積み上がります。全体の速さを決めている工程を**制約**と呼び、そこから改善する考え方が**制約理論**です。



この例では、レビューできる件数が変わらない限り、利用できる状態になるのは一日5件までです。実装済みのPRが増えるほど、着手から利用可能になるまでの時間は長くなります。数字は仕組みを説明するための例です。実際の制約は、工程ごとの件数と待ち時間から確かめます。

次の工程が受け入れられる量を超えて仕事を渡すと、待つ案件が増える。
実装を速めた効果が届けるには、レビューへ渡す量と、確認できる量も揃える。



レビュー待ちは、詰まる理由に合わせて減らす

一日10件を作っても5件しか確認できない例では、まず、なぜ5件で止まっているかを調べます。レビューを急がせるだけでは、必要な確認を削ってしまうかもしれません。

確認する時間がない

新しい実装への着手を絞り、レビューする時間を確保する。担当者へ依頼が集中していないかも見る。

差分の意図が読めない

変更を目的ごとに分け、判断理由とテスト結果を添える。確認のたびに説明を聞き直す時間を減らす。

何を正解とするか未決定

コードを読む前に、期待する動作と決める人確かめる。判断待ちをレビュー担当者の遅さにしない。

変えた後は、待ち時間だけでなく、本番へ届いた件数と手戻りも見ます。レビューが速くなっても、次に本番反映で待っていれば、そこを改めて調べます。

忙しい人を探すのではなく、仕事が進まない理由を取り除く。



制約が実装にある現場もある

前の例ではレビューが制約でした。しかし、どの工程が制約かは、人数や役割の数だけでは決まりません。直近の仕事がどこで列になり、次の工程が何を待っていたかを見ます。

実装が制約になっている状態

作る内容は決まり、レビューする人も待っている。それでも、実装を待つ案件が増えている。ここなら、AIで実装時間を短くすると全体も速くなりやすい。

判断やレビューが制約になっている状態

実装は終わっているのに、優先順位、仕様、レビュー、リリースの判断を待つ案件が増えている。ここでは、実装だけを速めても全体は速くならない。

バリューストリームマッピングで直近10件を作業と待ちに分け、待ちが集中する工程と、その手前に積み上がる件数を見ます。制約が分かったら、そこへ入れる仕事を絞る、不要な確認を減らす、必要な人や時間を増やす、という順で対処します。

AIをどこに使うかは、いまの制約を動かせるかで決める。



独立したバリューストリームの四つの条件

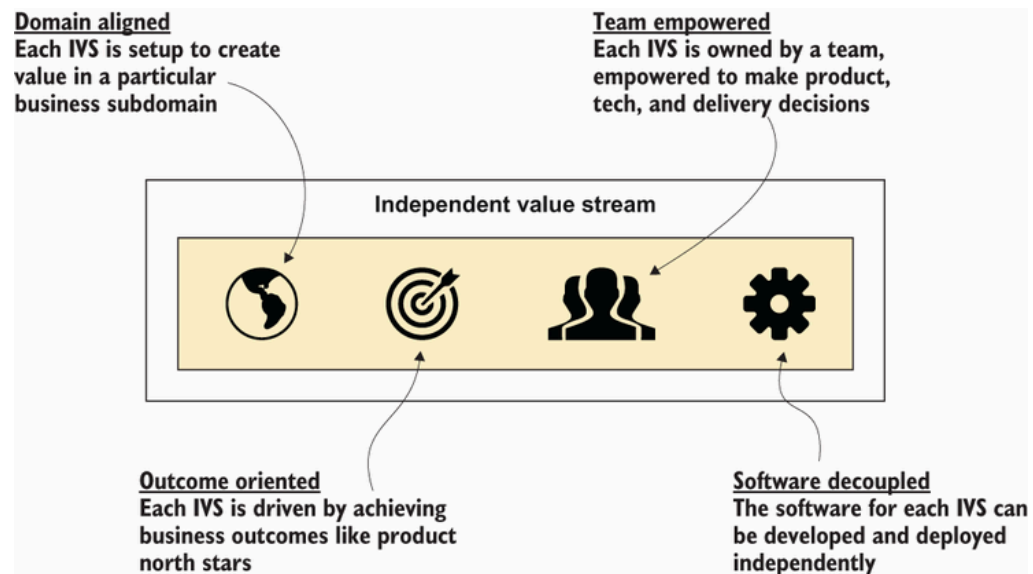


Figure 1.4 Independent value stream より引用

制約が判断や引き継ぎにあるなら、チームが流れをどこまで自分たちで進められるかを見直します。ここでいう独立とは、関係するチームがあっても、日常的な変更を発見から結果の確認まで大きな待ちなしに進められることです。そのために、チームの担当範囲、判断できる範囲、目標の置き方、ソフトウェアの分け方という四つの条件を揃えます。

事業の仕事に合わせて担当範囲を決める どのジョブを扱うかが明確

チームが判断できる 製品・技術・リリースを決められる

利用後の変化から作るものを決める 届けた結果まで確かめる

ソフトウェアを分ける 単独で変更して届けられる

この四つは別々ではありません。担当範囲と目指すアウトカムが決まっても、判断のたびにほかのチームを待ち、ソフトウェアも一緒に変更しなければならないなら、流れは独立していません。

四つの条件は、「何を担うか」と「どう進めるか」

先ほどの四つは、「何を担うか」と「どう進めるか」の二つに分けて考えます。事業の仕事と目指すアウトカムは、チームが引き受ける範囲を決めます。判断できる範囲とソフトウェアの分け方は、その仕事を大きな待ちなしに進められるかを決めます。

何を担うか

事業の仕事に合わせて担当範囲を決める

「注文」「決済」のように、事業の中で一つの役割を担う範囲へ集中する。関係の薄いジョブを同じチームへ集めない。

利用後の変化から作るものを決める

機能の完成ではなく、利用後に起きてほしい変化を目標にする。届けた後の結果まで同じチームが確かめる。

どう進めるか

チームが判断できる

合意した予算・品質・権限の範囲で、何を作り、いつ届けるかを決める。範囲を超える変更は、相談先を先に決める。

ソフトウェアを分ける

ほかのシステムを同時に変えなくても、開発・テスト・リリースできる。依存があっても、接続方法やデータ形式を安定させる。

四つの条件を揃えるのは、チームを閉じるためではありません。ジョブの発見からアウトカムの確認までを、大きな待ちなしに進めるためです。



分ける単位は、一緒に変更しなくて済む範囲で考える

ソフトウェアを小さく分けても、変更のたびに他チームの修正が必要なら、待ちは残ります。境界を決めるときは、ファイルやサービスの数より、どこまで自分たちで変更を完結できるかを見ます。

検索側で変えられること

商品データを受け取る形式が安定していれば、検索の内部処理を変えても、商品管理側の同時改修を避けやすい。

一緒に決める必要があること

納期や在庫の意味、データの持ち主、更新の反映条件を変えるなら、使う側と提供する側で確認が必要になる。

細かく分けるほど接続や運用の仕事も増えます。頻繁に変える部分から、データの責任と接続の取り決めに明確にします。分割や全面刷新そのものを目的にはしません。

境界の良し悪しは、箱の小ささより、変更に必要な調整で確かめる。



サービスを分けても、共有データの変更待ちは残る

検索と商品管理を別のサービスにしても、同じテーブルの内部構造に依存していれば、自由には変更できません。**データアーキテクチャ**では、データの形、保存場所、渡し方、更新する責任を考えます。これは、チームがどこまで独立して変更できるかにも関わります。

同じテーブルを直接読む

検索、商品管理、帳票が同じ列を使う。列の名前や意味を変えるたびに、どの処理が壊れるかを調べ、関係者の確認を待つ。

提供する情報と変更の責任を決める

商品管理が更新を担い、検索や帳票へ必要な情報を渡す。提供する形式と意味を保てる変更なら、内部の同時改修を避けやすい。

ただし、分ければ同期や障害対応の仕事も増えます。一緒に更新しないと矛盾するデータは、同じ場所で管理する方が簡単な場合もあります。まず共有している項目と利用者を調べ、調整が多い箇所から見直します。

DBを分けることが目的ではない。誰が何を変え、誰へ知らせるかを明確にする。



2. 測りやすい数字は、価値が届いた証拠ではない

変化を確かめるには、作った量と、利用後に起きた変化を分けて測る。

測りやすい数字はAIで伸びる

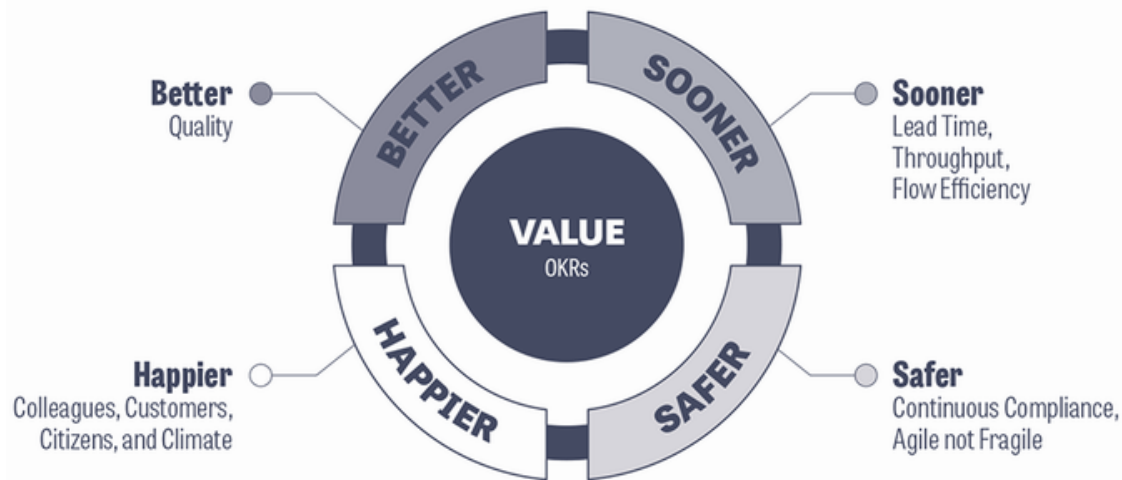
PR数、コード変更の記録（コミット）の数、変更行数。どれもダッシュボードに出しやすく、AIを入れると増やししやすい数字です。ただ、これらが表すのは**アウトプットの量**です。たとえば、データベース内で動く処理（ストアドプロシージャ）の作成数を個人目標にすると、必要性より作成数が優先されます。PR数を個人目標にしても、アウトプットそのものが目的になる同じ問題が起きます。

AIで増えやすいのはアウトプット。価値を判断するにはアウトカムを確かめる。

頼まれた機能を作り続け、アウトカムよりアウトプットの量と出す速さだけを見るチームを**フィーチャーファクトリー**と呼びます。機能を作ること自体が目的になり、使われた結果を確かめない状態です。

アウトプットで仕事の流れを、アウトカムで利用後の変化を確かめる。

価値は「速く」だけでは測れない



変更を良くしたかは、速さだけでは決まりません。品質、アウトカム、速さ、安全性、関わる人の満足を一緒に見ます。

AIで作る時間が短くなっても、手戻りや障害が増え、レビューする人の負担が重くなれば、改善したとは言いきれません。

Figure 1.3 Better Value Sooner Safer Happier (Source: Smart et al., Sooner Safer Happier [IT Revolution, 2020]) より引用

「早くなったか」だけでなく、「何が良くなり、どこに負担が移ったか」を見る。

出典：Nick Tune, Jean-Georges Perrin, *Architecture Modernization*, Manning, 2024 / Jonathan Smartほか, *Sooner Safer Happier*, 2020

同じ変更を、五つの面から確かめる

五つは、同じ変更を別の面から見るための観点です。速さを上げた結果、品質や安全性、関わる人の満足を悪化させていないかを確認めます。

Better 期待した動作を安定して行えるか。品質は上がったか

Value ユーザーのジョブが片付き、事業のアウトカムが変わったか

Sooner 小さな変更を、早く、繰り返し届けられたか

Safer 変更による失敗と、起きたときの影響を減らせたか

Happier ユーザーと、開発・運用に関わる人の負担を減らせたか

五つを一つの点数にまとめず、どこが良くなり、どこが悪くなったかを並べて判断する。



アウトプットを届ける速さを三つに分ける

五つのうち、まず**Sooner**を測ります。アウトプットを届ける流れは、一つの数字だけでは分かりません。かかった時間、届けた件数、その時間のうち実際に作業していた割合を分けて見ます。

リードタイム

作業に着手してから、変更を利用可能にするまでの**経過時間**

スループット

1週間などの一定期間に、利用できる状態まで進んだ変更の**件数**

流れの効率

リードタイムのうち、実際に手を動かした**時間の割合**

30/70の例なら、流れの効率は30%。残り70%は待ち時間。

この三つで分かるのは「アウトプットを届ける速さ」です。使われた結果であるアウトカムは、別に確かめます。



アウトプットの速さとアウトカムを並べて見る

リードタイムやスループットで分かるのは、アウトプットを届ける速さです。価値まで見るには、**同じ期間と対象ユーザー**について、利用とアウトカムを並べます。検索条件を追加した例なら、次の三つです。

アウトプットを届ける流れ

着手から利用可能になるまでの時間と、そのうち誰かを待っていた時間

利用の手がかり

対象ユーザーが新しい検索条件を使った割合と、検索を途中でやめた割合

アウトカム

必要な情報へたどり着くまでの時間と、その後に購入へ進んだ割合

アウトプットを早く届けたことと、期待したアウトカムが起きたことを、別々に確かめる。

「選べたか」は、操作の記録だけでは分らない

検索条件を使った記録は残せます。しかし、利用者が必要な情報を見つけ、納得して商品を選べたかまでは分かりません。先ほどのアウトカムを確かめるには、操作の記録と利用者への確認を組み合わせます。

記録で確かめること

検索を始めた時刻、条件の使用、候補を選ぶまでの操作、途中でやめた割合。購入へ進んだかも、対象と期間をそろえて見る。

本人に確かめること

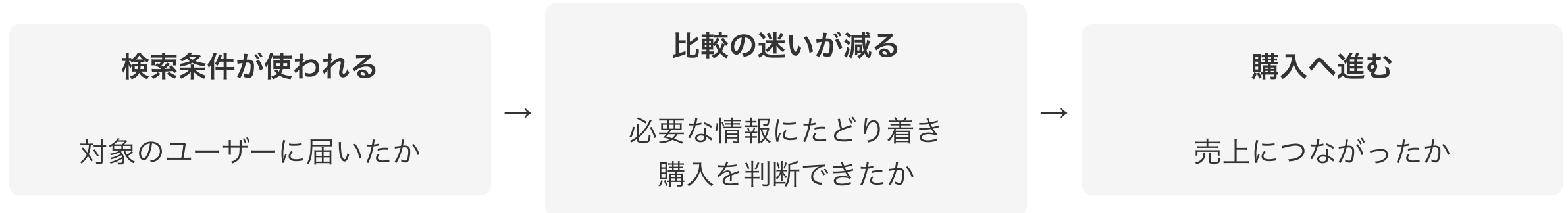
どの情報で決められたか。まだ何が足りなかったか。購入しなかったのは、比較できなかったからか、条件に合う商品がなかったからか。

短時間で離れた人を「早く選べた人」と数えないようにします。選べた人だけの時間を比べると、選べずに離れた人を見落とします。件数が少ない場合は、確かな傾向とは言い切らず、分かったことと未確認のことを分けます。

操作が終わったことと、目的を果たせたことを区別する。

購入が増えた理由を、変更と結びつけて確かめる

検索条件を追加した後に購入が増えても、その機能が理由とは限りません。同じ時期の値引きや、訪れたユーザーの違いでも数字は変わります。まず「どう効くはずだったか」を分けて考えます。



条件を使っても迷いが減らなければ、画面や情報を見直します。迷いが減っても購入されなければ、価格や在庫など、次の理由を調べます。可能なら同時期の新旧画面を比較し、難しければ利用者への確認とほかの変化の記録を併せて判断します。

売上だけで成功・失敗を決めず、期待した変化がどこまで起きたかを見る。

作る速さだけを上げると、使われない機能も積み上がる

顧客のジョブよりスケジュールを優先して機能を量産する状態を、Melissa Perriは**ビルドトラップ**と呼びました。アウトプットの量だけをAIで増やすと、この罠に入りやすくなります。多くのチームは、新しいものを足すのは得意でも、古いものを消すのは苦手だからです。かつて入ったプロジェクトでは、データベース内で動く処理を調べたところ、約30%がすでに使われていませんでした。

機能というアウトプットが1つ増えるたびに

レビューで読む範囲、テストの対象、運用で守る対象、ユーザーが比べる選択肢が増える。

増えた複雑さが、次の変更を遅くする

複雑さは、コードの行数よりも、安全に変更するために把握すべきことの多さとして表れます。古い設定や依存、テストが残っているほど、判断とレビューに時間がかかります。

古いものを減らさず生成だけを増やせば、次の変更は遅くなる。



速くなった体感も、測定の代わりにならない

AIの影響を調べる研究組織METRは、経験豊富なオープンソースソフトウェア（OSS）開発者16人に、普段扱うコード群の246課題をAIツールあり・なしで解いてもらい、所要時間と本人の体感を比べました。

実測

各課題は、AIツールを使える条件と使えない条件へ無作為に割り当てられました。2025年前半のツールを使える条件では、完了までの所要時間が**推定19%長**くなりました。測ったのは生成にかかる時間だけでなく、確認や修正を含む課題の完了までです。

体感

事前には「24%速くなる」と予想し、終わった後も「20%速くなった」と感じていた。測定と体感が、逆向きだった。

大規模で成熟したコード群に詳しい開発者と、2025年前半のツールを対象にした結果です。現在のすべての開発に当てはめる数字ではありません。ここで分かるのは、**本人の体感と実測が逆向きになる場合がある**ことです。自分たちの流れを測って判断待ちが見えたら、次の問いは「なぜ、その判断が止まるのか」です。

PR数や体感だけで決めない。バリューストリームマッピングで流れを測り、アウトカムと並べて見る。



3. 役割ごとの前提を対話で確かめる

待ちの場所が分かってても、役割ごとに問題の見え方が違えば、直し方は決まらない。

役割が違えば、同じ機能でも見え方が変わる

購入前の相談から開発までに関わる役割を例にします。営業、PM（プロダクトマネージャー）、デザイナー、エンジニアは、同じ機能について別の情報を持っています。

営業

顧客との会話
購入の条件

PM

目指す結果
優先順位

デザイナー

利用する場面
操作のつまずき

エンジニア

コードと依存
レビューとテスト

情報が別々の資料や会話に散らばっているだけなら、一か所へ集めれば解決します。難しいのは、同じ情報を見ても、役割が背負う責任によって**何を問題と考えるか**が変わることです。

情報を揃えるだけで、判断の前提まで揃うとは限らない。

「違う情報」の奥に、違う解釈がある



宇田川元一『他者と働く』

宇田川元一さんは、物事をどう理解し、何を正しいと考えるか、その**解釈の枠組み**をナラティブと呼びます。立場、経験、背負っている責任が違えば、「今月リリースするか」という同じ判断でも、心配することが変わります。

営業 延期すると、商談の機会を逃す

デザイナー 今の画面で出すと、利用者が途中で迷う

エンジニア 応急処置で出すと、次の変更が難しくなる

SRE（信頼性と運用を担う役割） 復旧手順がないまま出すと、障害対応が長引く

どれか一つだけが正しいとは限りません。相手の判断が不合理に見えるときほど、その役割から何が問題に見え、何を守ろうとしているかを確認めます。

「相手が分かっていない」と決める前に、相手からは何が問題に見えているかを確認める。

出典：宇田川元一『他者と働く——「わかりあえなさ」から始める組織論』2019 / [NewsPicks Publishing](#)

仕組みで解ける問題と、対話が要る問題を分ける

『他者と働く』では、既存の知識を当てはめられる**技術的問題**と、関係者が問題の捉え方から見直す**適応課題**を分けます。ソフトウェア開発の待ちには、両方が混ざっています。

技術的問題

原因と目標の認識が揃っており、既存の手順や専門知識を使える。たとえば、ビルド時間をキャッシュで短くする。

適応課題

何を問題とみなすか、何を優先するかが役割ごとに違う。一つの役割だけで解決策を決めても、仕事の進め方は変わりにくい。

承認待ちのすべてが適応課題ではありません。重複した確認は仕組みで減らせます。一方、期日、使いやすさ、変更の難しさ、障害リスクのどれを優先するかで止まっているなら、先に互いの前提を確かめます。

手順の問題は仕組みで減らす。前提の違いから生じる待ちは、対話から始める。



対話は、同意を急ぐことではない

ここでいう対話は、説得や情報共有の言い換えではありません。互いを、指示に従わせる相手ではなく、問題と一緒に扱う相手として関係を作り直すことです。『他者と働く』では、次の四つを行き来しながら進めます。

準備

分かり合えていないと認め、自分の前提をいったん保留する

観察

相手の言葉、行動、置かれた状況から、何を守ろうとしているかを見る

解釈

相手の立場なら、なぜその判断が合理的なのかを考える

介入

双方が動ける小さな働きかけを試し、反応を見てまた観察する

一度で分かり合うための順番ではありません。相手の反応から見立てを直し、次の働きかけを変える反復です。まず会話の例で確かめ、その後で対話の材料を図に並べます。

図は結論を出すためではなく、違う前提を出し、次に確かめることを決めるために使う。



「今月出したい」の理由を、役割ごとに聞く

たとえば、営業は「今月出したい」、エンジニアは「まだ出せない」と言っているとします。期日の賛否を繰り返す前に、それぞれが守りたいことを確かめます。

営業に確かめる

「今月必要なのは、誰のどんな判断ですか」
→ 顧客が導入を決めるために、商品を比較できることを確かめたい。

エンジニアに確かめる

「どんな失敗を心配していますか」
→ 全顧客へ公開すると、現在の検索へ戻せない変更が含まれる。

この場合、対象の顧客とデザイナーを交えた試用で、購入判断に必要な情報を確かめる案が考えられます。それで営業の目的を満たせるかを聞き、本番公開の範囲と戻す方法は別に決めます。

前提を確かめると、話し合う対象が「出す・出さない」から、
「誰に、何を、どこまで確かめてもらうか」へ具体化する。



対話の材料を、ワードリーマップに置く

ユーザーが得たい結果、その結果に必要な要素、各要素の成熟度を一枚に並べると、どこへ投資し、どう運用するかを比べられます。この図を**ワードリーマップ**と呼びます。

ジョブ

誰が、どんな状況で、何を達成したいか

要素同士の依存

待ち時間ではなく、ジョブを満たすために何が何を必要とするか

成熟度

作り方や市場の選択肢が、どれだけ確立しているか

ワードリーマップでは、**ユーザーのジョブを何が支え、各要素にどんな選択肢があるか**を見ます。どこで試し、どこへ投資し、何を置き換えるかを話すための図です。後で扱うC4モデルは、ソフトウェアの構造を詳しく見るために使います。見る目的が違うので、一方をもう一方の詳しい版とは扱いません。

要素だけでなく、「誰のため」と「どの段階」を同時に見る。



縦は見えやすさ、横は成熟度

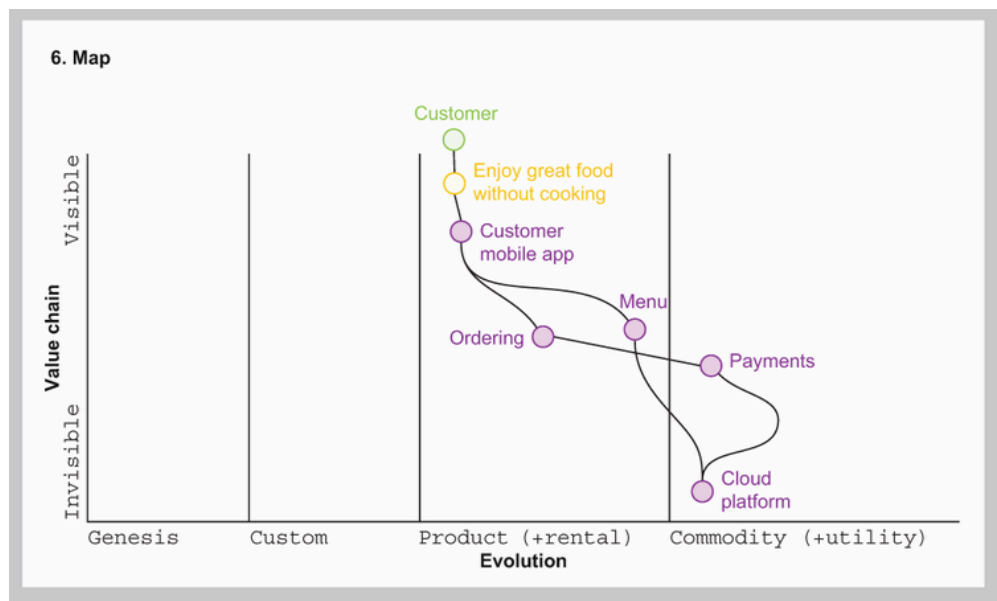


Figure 5.7 Step 6 of the Wardley Mapping Canvas, a Wardley Map より引用

縦はユーザーからの見えやすさ（可視性）です。一番上にユーザーとジョブを置き、そこから下へ、ジョブを満たすために必要な要素を線でつなぎます。線は、上の要素が下の要素を必要とする関係です。上にあるほどユーザーから見えやすく、下にあるほどユーザーからは見えにくい内部の要素です。

横は成熟度です。左から、まだ答えを探している段階、独自に作り込む段階、製品として選べる段階、電気のように誰もが使える共通基盤へ進みます。

横軸は優劣を表しません。置く位置によって、試行錯誤する、独自に作る、製品を選ぶ、共通基盤として扱うなど、適した進め方が変わります。

縦で「なぜ必要か」を確認し、横で「どう扱うか」を考える。

成熟度は、自社の実装年数では決めない

ウォードリーマップの横軸は、古い技術か新しい技術か、自社で何年使ったかを表すものではありません。その要素の作り方や利用方法が、市場でどれだけ知られ、選べるようになっているかを見ます。

まだ答えを探している要素

どの検索条件が顧客の判断に役立つか、まだ確かめられていない。要望だけで決めず、試用や観察を重ねる。

既製の選択肢がある要素

検索エンジンや保存領域について、機能・性能・運用条件を比較できる。ただし、既製品が自分たちの必要条件を満たすかは別に確認する。

位置に意見が割れたら、候補となる製品、実際の利用例、満たせない条件を持ち寄ります。根拠がない位置は仮置きとします。成熟した要素でも、自前運用が適する場合があります。

横軸は結論ではない。何を調べ、どの選択肢を比較するかを決める材料です。



検索の例で、図の読み方を確かめる

検索機能の例を置くと、縦方向にはジョブを支える依存が見え、横方向には要素ごとに適した進め方が見えてきます。

ジョブから依存をたどる

必要な情報へ早くたどり着きたい



検索画面



商品データ・検索エンジン



計算資源・保存領域

成熟度で進め方を変える

どの検索条件が役立つかは、ユーザーと小さく試す。検索エンジンは、既製サービスと自前運用を比べる。計算資源や保存領域は、共通基盤として扱えるかを確かめる。

一つの機能でも、すべてを同じ方法で作る必要はありません。ジョブに近く、まだ答えがない部分には試行錯誤の時間を使います。選択肢が確立した部分は、必要な制御、運用の負荷、総費用で選びます。

どこで試し、どこで既製の選択肢を使うかを、要素ごとに決める。



同じ図に、役割ごとに見ている情報を置く

検索機能を例に、PM、デザイナー、エンジニア、SREが持つ情報を、同じワードリーマップに並べます。役割が違えば、確かめたいことも変わります。

PM

誰のどのジョブか
何が変われば価値か

デザイナー

どこで利用者が迷うか
どんな使われ方をするか

エンジニア

どの要素が必要か
変更がどこへ影響するか

SRE

自前で運用すると何が増えるか
障害がどこまで影響するか

誰のジョブを、何が、どの成熟度で支えるかを一枚で確かめる。



差別化だと思っていたものが、右端へ動いていた

私たちが差別化の中心（コア）だと信じて作り込んでいた領域が、あるときクラウド事業者が運用するマネージドサービスと呼ぶ数十行のコードに置き換わりました。

「自分たちで作っている」と「まだ独自に作る必要がある」は別のことです。自分たちが独自だと思っても、市場には既製の選択肢が増え、ユーザーも特別な機能とは見なくなることがあります。ウォードリーマップでは、こうした変化を右への移動として表します。

横軸の右端では、まず既製サービスと比べ、自前運用を続ける理由を問い直します。ただし、必要な制御、性能、障害の切り分け、法令対応、総費用が合わなければ、自前運用へ戻すこともあります。

右端は「買え」ではなく、既製サービスと自前運用を比較し直す合図。

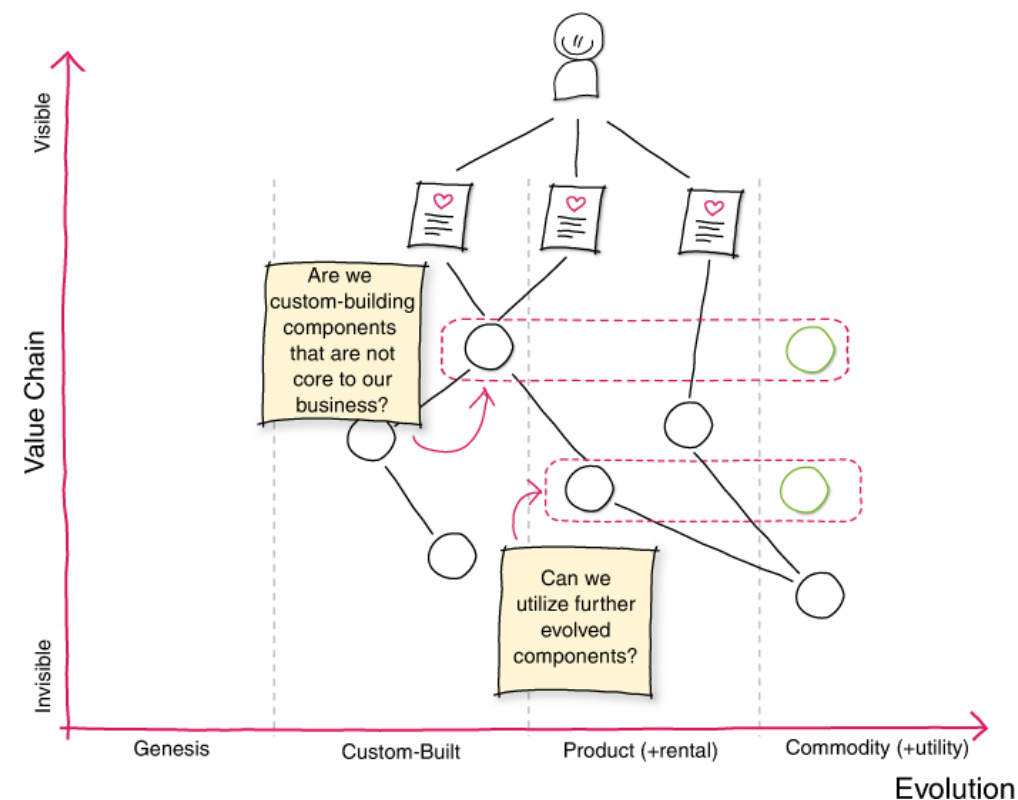


Figure 1.10 Are we investing efficiently? (Architecture for Flow) より引用

出典：Susanne Kaiser, *Architecture for Flow* / Nick Tune, Jean-Georges Perrin, *Architecture Modernization*

半年計画の刷新は、ジョブを確かめずに始めた

半年かける予定だった大規模刷新を、4ヶ月目にロールバックしました。つまり、変更を取り消して元に戻しました。振り返ると、「誰のどのジョブを片付けるのか」を問わないまま、**作れるものを作っていた**。

ワードリーマップで見ると

この半年計画では、刷新の対象をシステム内部の変更に置き、ユーザーが何に困っているかを確かめていませんでした。そのため、変更がどのジョブに結びつくかを説明できませんでした。

バリューストリームで見ると

ロールバックで、4ヶ月分の変更は誰のジョブも片付けずに消えた。作れることは分かりました。しかし、作るべきだったかは最後まで確かめられませんでした。

4ヶ月分を取り消すとなると、やめる判断そのものが重くなります。最初の判断が間違いだったように見えることも、決断を遅らせます。判断が疑われるまでの時間が長いほど、直す費用は高くなる。やめる判断を早くする方法は、このあと扱います。

ユーザーのジョブとの関係を説明できない計画は、始める前に問い直せたはず。



刷新では、古いコードが守っていた動作も引き継ぐ

刷新の目的を決めても、書き直すだけでは元のシステムを引き継ぎません。長く使われたコードには、利用者の例外的な使い方や、過去の障害への対処が残っています。理由が分からない処理を、不要だと判断するのは早すぎます。

たとえば、検索結果が同じでも

更新の反映が遅くなれば、販売終了の商品を案内するかもしれない。エラーの返し方が変われば、呼び出し元が再試行できなくなるかもしれない。

置き換える前に残す

守る動作はテストへ。そう決めた理由は設計の記録へ。実際の利用や障害の記録と照らし、必要な動作と古い都合を分ける。

すべての古い動作を永久に残す必要はありません。変えるなら、誰が依存しているかを確かめ、移行方法を決めます。後半では、C4モデルという構造の表し方を使って、その影響を調べます。

**残すべきなのは、利用者が頼っている動作と、その理由。
それを確かめられて初めて、実装を変える選択肢が増える。**



図に並べると、何を話し合うかが揃う

PM・デザイナー・エンジニアに、ユーザーと直接話す営業やCS（カスタマーサクセス）も加わって30分だけ集まり、ユーザーのジョブ、必要な要素、依存をざっくり並べます。最初から正確でなくても、見ている情報の違いが分かれば話し合いを始められます。

ジョブとの関係を説明できない要素

今回の範囲外。捨てずに、別のジョブとの関係を確認め直す

横軸の右端にある要素

既製サービスと自前運用を比較。必要な制御と総費用で決める

依存の線が集まる要素

複数の機能や仕組みが共通して必要とする要素。バリューストリームマッピングで測った待ち時間と見比べ、担当するチームを決める

図の目的は、関係者全員が見ている情報を揃え、投資を決める理由をはっきりさせることです。最初から現状の細部まで正確に描く必要はありません。まず、合意にかかる時間を短くします。次に、**ジョブの発見から結果の確認までを、どのチームが担当するか**を見直し、チーム間の引き継ぎそのものを減らします。

図で意見の違いを具体化する。計測で、次に確かめることを選ぶ。



判断待ちを短くする三つの取り決め

前提を確かめる対話と並行して、判断の依頼そのものにも形を与えます。判断が止まりやすいのは、誰が決めるのか、何をいつまでに返すのか、返事がないときにどう進めるのかが曖昧なときです。

決める人を一人にする

その判断を引き受ける権限と責任のある人を決める。詳しい人や影響を受ける人から、判断材料を集める。

全員一致を待たない

反対意見は記録に残す。決まった後は、その方針で動く。

依頼に四つを書く

誰に、何を、いつまでに、返事がないときはどうするか。「なるべく早く」は期限ではない。

判断を目的に集まるなら、決められる人を呼び、必要な情報を先に渡します。進捗共有は文書で済ませ、前提の食い違いが大きいときは対話に時間を使います。10人で1時間話すなら、その10人時で何を確かめるかを明確にします。

判断の依頼は、内容だけでなく「誰が・いつまでに・返事がないとき」を決める。

返事がないことを、承認とは扱わない

判断を待ち続けないためには、期限だけでなく、その後の動き方が必要です。ただし「返事がないければ進める」を、あらゆる変更に適用すると、必要な確認まで省いてしまいます。

確認できるまで止める変更

本番データの削除、閲覧権限の変更、元に戻せない移行など。期限を過ぎたら代わりに判断できる人へ相談し、承認済みとはみなさない。

合意した範囲で進められる作業

検証環境での比較や、利用者へ影響しない調査など。あらかじめ決めた範囲なら進め、結果を報告する。範囲を超えたら再度相談する。

「金曜までに判断できなければ、本番公開は保留し、試用の準備だけ進める」のように書きます。全員一致を待たないことも、必要な専門確認や安全上の条件を省くことはありません。

待ちを減らすとは、確認を飛ばすことではなく、止める範囲と進める範囲を決めること。



計測と図を、日々の判断へつなぐ

長い待ちや前提の違いを見つけても、眺めているだけでは流れは変わりません。何が一番の問題かを絞り、どこへ力を集めるかを決め、日々の仕事へ反映します。

診断

いま何が起きているか

実際の記録と各役割の見方から、全体の流れを最も止めているものを一文で説明する。

方針

何を選ぶか

どのジョブとアウトカムを優先し、いまは何をやらないかまで決める。

進め方

どう続けるか

決める人、判断日、確認する数字、例外の扱いを、日々の流れに組み込む。

「改善する」だけでは進めない。何に力を集め、いまは何をしないか、どう確かめるかまで決める。



4. ジョブから「作る・任せる・やめる」を判断する

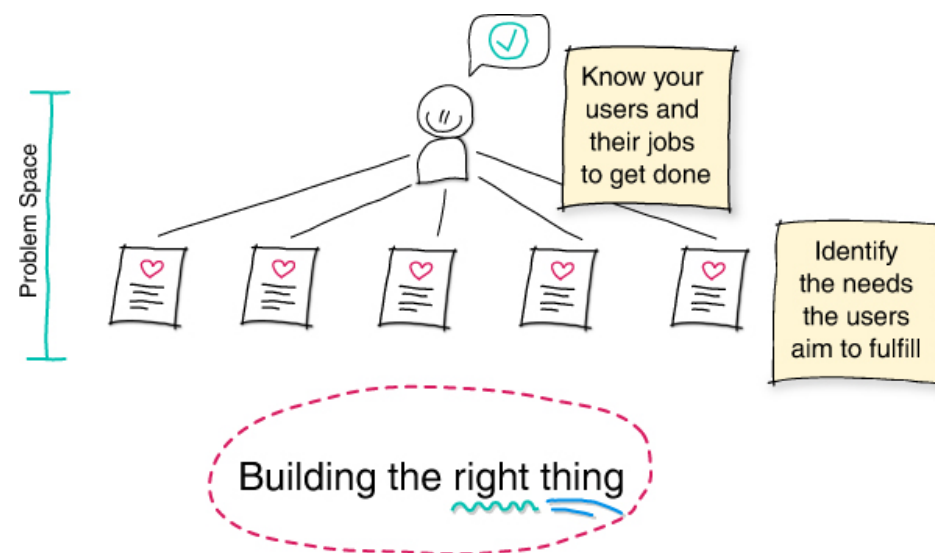
作る理由を決め、確かめられる範囲をAIへ任せ、やめる条件と判断日を残す。

ジョブは、解決策が変わっても残る

ジョブ理論では、ユーザーは目的を果たすためにプロダクトを「雇う」と考えます。機能が揃っていてもジョブが片付かなければ、別の手段へ乗り換えます。

状況とジョブが同じなら、**解決策を変えてもジョブそのものは変わりません**。AIで作れるものは増え続けますが、ユーザーのジョブは、私たちの実装能力に合わせて増えません。

作れるものがいくら増えても、ユーザーのジョブは増えない。



"People hire products or services to get a job done."

Clayton Christensen

Figure 1.1 Starting from the user perspective to build the right thing (Architecture for Flow)
より引用

出典：Christensen Institute, Jobs to Be Done Theory / Susanne Kaiser, Architecture for Flow

作るべきかは、四つの問いで決める

四つの問いで、作る理由、実際の困りごと、期待するアウトカムと確かめ方、代わりの手段を確認します。実装案を比べる前に、そもそも作る必要があるかを判断します。

作るものとジョブの関係を説明できるか

手段を消し、「どんな状況で、何を達成したいのか。なぜそれが大切か」を一文で書く

最近、実際に困った場面があったか

抽象的な賛否より、直近の行動を見る。そのとき何を試し、何が決め手だったか

期待するアウトカムと、確かめ方を決められるか

利用後に何が変われば価値が届いたと言えるか、いつ、誰が確かめるかを先に決める

既存の手段や小さな実験で、先に確かめられないか

まず既製サービスと比べる。必要な制御や総費用が合わなければ自前で持つ。不確かなら小さく試す

答えられない問いがあれば、実装を始める前に、ユーザーの行動や実際の待ち時間を確かめる。



書かなかった判断は、実装の中で補われる

作る理由を決めても、依頼に書かなければ伝わりません。AIは依頼文や既存コードから不足を補って進めることがあります。途中で確認しないまま複数の作業を進めると、異なる前提で実装されても、レビューまで気づけません。

架空の依頼

「検索APIを速くする」



案A 問い合わせを改善し、今の結果を保つ

案B 結果を再利用し、更新の反映が遅れる

案C 返す件数を減らし、取得時間を縮める

同じ依頼から考えられる案ですが、利用者への影響は違います。情報の古さや結果の件数を変えてよいかが未決定なら、速くなっただけでは受け入れられません。問題は実装が違うことではなく、許容する変更をチームが決めていないことです。

実装方法の違いと、守る条件の食い違いは別です。
決めていないことを、相談なしに決まったことにしない。



曖昧さを残すと、レビューが仕様を決める場になる

先ほどの案Bが完成してから「納期が古くなるのは困る」と分かったら、コードの確認だけでは済みません。許せる遅れを関係者へ聞き、方式を選び直し、実装とテストを直すことになります。

差分から前提を探す

何を変えてよいと
判断したのか

→

関係者へ確認する

その動作で
ユーザーが困らないか

→

実装し直す

変更範囲とテストを
組み直す

複数のPRが同じ未決定事項に依存していれば、その判断が終わるまで全部が待ちます。AIがコードを書く時間を短くしても、ここで増えた調査・相談・手戻りは、そのままでは短くなりません。

文書を書くのは、説明を増やすためではない。
実装後に発覚すると高くつく判断を、先に済ませるためです。

実装前の判断を、設計文書やPRに書く

設計の背景と方針を書く Design Docや、PRの説明に、AIが作業前に読む判断材料を置きます。この発表では、期待する動作と守る条件をまとめたものを**仕様**と呼びます。七つの観点を使って、決め忘れを探します。

目標 誰のどのジョブを、なぜ解くか
受け入れ基準 結果を見て判定できる条件

不変条件 変えてはいけない動作や品質
境界 対象と対象外。越えるなら止める

完了の定義 テスト、文書、確認結果など、PRを受け入れられる状態

先行事例 似た実装と、過去に試して失敗した案
未解決の問い 決める人と、最初の調べ方

七つを毎回すべて埋める必要はありません。小さな変更はPRの説明へ、複数のPRにまたがる変更はDesign Docへ書きます。文量は、一回の作業で判断する範囲に合わせ、背景は必要な箇所へリンクします。

長く詳しく書けばよいわけではない。曖昧な言葉、隠れた前提、古い情報を減らし、一度で読み切れる量に絞る。



「検索を速くする」を、確かめられる依頼へ変える

先ほどは検索APIの速さを例にしましたが、本当に短くしたいのは、商品を選ぶまでの時間かもしれません。ここでは、候補を絞れず困っていると分かった架空例を考えます。利用者の困りごとと変更の条件をまとめ、効果は仮説として書きます。

目標と仮説

候補が多く比較できない人を助けたい。納期で絞れば、購入候補を選びやすくなると考える。

受け入れ基準

指定した納期に合う商品を表示し、条件を外すと元の結果に戻る。該当なしの場合も表示する。

守る条件と変更範囲

閲覧権限は変えない。今回の対象は絞り込み。検索基盤の交換が必要なら、着手前に相談する。

未決定の点と完了の確認

納期末定の商品を含めるかはPMが確認する。PRにはテスト結果と実際の操作の記録を添える。

実装の受け入れ条件と、利用後に確かめる効果を分ける。
テストが通ったことだけで、「購入候補を選びやすくなった」とは決めない。



決めきれないことは、実装ではなく調査として任せる

最初からすべてを決める必要はありません。納期で絞れると本当に選びやすいのか、今の基盤で実現できるのかが分からないなら、まず確かめる作業を依頼します。採用する実装を作る仕事とは、完了の条件を分けます。

調査・比較を任せる

遅い処理の計測、既存コードの確認、案ごとの効果と費用の比較を求める。仮定と未確認の点も報告してもらう。本番へ反映しない。

実装を任せる

採用する案と守る条件を決める。権限や結果の意味を変える必要が出たら、推測で進めず相談する。実装方法の細部は提案してもらう。

複数のAIに異なる案を比較させること自体は役立ちます。その場合も、比較する条件と試す範囲をそろえます。調査で前提が変わったら依頼を更新し、着手済みの作業へ変更点を伝えます。

未決定をなくしてから頼むのではなく、未決定を勝手に確定させない頼み方をする。



AIには、実装より先に計画を出してもらおう

仕様で「何を満たすか」を決めたら、「どう進めるか」はAIに提案してもらいます。人は、その計画が目標、受け入れ基準、不変条件、境界を外していないかを確認します。実装を始めるのは、その後です。



計画が仕様から外れている

読み違い、情報不足、実現上の制約を確かめます。必要な情報を補うか、変更が必要な条件を相談します。

計画は仕様どおりだが、望ましくない

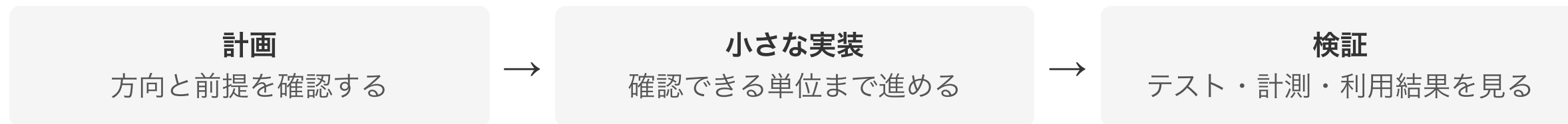
仕様に必要な条件が抜けているか、選んだ方法に問題がないかを確認します。ジョブやアウトカムへ戻り、依頼と計画のどちらを直すかを決めます。

数百行の差分から意図を推測する前に、計画の段階で方向のずれを止める。



計画を読んだだけでは、検証したことにならない

計画の確認で分かるのは、目的へ向かう道筋が妥当かどうかです。実際に正しく動くか、安全か、アウトカムにつながるかは、成果物を動かして初めて分かります。流暢で詳しい計画も、その証拠にはなりません。



大きな作業を一度に実装すると、最初の読み違いが後続の変更へ広がります。調査、構造を決める変更、動く最小単位の実装へ分け、区切りごとに結果を確かめてから次へ進みます。

計画は方向をそろえる。検証は、実際に起きたことを確かめる。両方を混ぜない。

テストが通っても、期待する動作が違えば困る

AIが実装とテストを両方作る場合、同じ読み違いが両方に入ることがあります。たとえば、納期末定の商品を検索結果へ含めるか決まっていなのに、実装でもテストでも「含める」としていれば、テストは通ってしまいます。

期待する結果の根拠を確かめる

チームが合意した動作、既存の外部仕様、過去の不具合を根拠にする。新しい実装が返した値を、そのまま正解にしない。

困る場面でも確かめる

納期末定、該当商品なし、閲覧権限なし、接続先が応答しない場合を試す。必要な動作を崩したら、検査が失敗するかも確認する。

別のAIを確認役にしても、同じ曖昧な依頼だけを渡せば十分とは限りません。何を根拠に正しいと判断したかを見ます。そのうえで、商品を選びやすくなったかは、テストとは別に利用者と確かめます。

検査の成功だけでなく、その検査が何を正しいと見なしているかを確認する。



AIに任せるのは、確かめて戻せる範囲まで

作るべきかをチームで決めた後も、AIへどこまで任せるかは一律ではありません。失敗をすぐに見つけれ、元に戻せる作業ほど広く任せられます。影響が大きく、誤りを見つけにくい作業ほど、小さく区切って人が途中で確認します。

仕様が伝えられるのは、チームが言葉にした意図と条件です。その変更がプロダクト全体にふさわしいか、ユーザーの負担に見合うかは、成果物を見て人が判断します。

広く任せやすい作業

ビルド、型検査、既存の動作を固定したテストで、成否を低い負担で確かめられる。失敗しても元に戻せる。

小さく区切る作業

権限、決済、本番データのように、誤りの影響が大きい。変更範囲を絞り、途中で人が確認する。

要約ではなく証拠を確認する

AI自身の説明をレビューの代わりにしない。可能なら作業を担当したAIとは確認役を分け、差分、テスト結果、計測、画面、分かっている不足を確かめる。

本番へ出す変更は、数か月後にも、何が変わり、なぜ安全だと判断したかを説明できる状態にします。

**AIへ任せる範囲は、誤りを見つけて元に戻せるところまで。
作業は任せても、作るべきかと受け入れるかは人が決める。**



任せる範囲と、同時に動かす数は別に決める

一つの作業をAIに任せられることと、複数の作業を同時に受け入れられることは別です。実装が並行して進んでも、判断とレビューが一人へ集まれば、そこで待ちが増えます。

一件を、どこまで任せるか

変更範囲、失敗のを見つけ方、途中で相談する条件を決める。難しい作業でも、検証環境で結果を確かめられるなら任せやすい。

何件を、同時に進めるか

作業同士が同じ判断やファイルを奪い合わないか、戻ってきた結果を確認できるかで決める。独立していなければ、先に順番を決める。

未確認の差分や未回答の質問が増え続けるなら、同時進行を減らします。ここでも見るのは、動かしているAIの数より、確認を終えて利用できる状態になった仕事です。

任せる量は、生成できる量と、チームが確認して引き受けられる量を見て決める。



同じ見落としは、コード・Lint・テストで防ぐ

AIが作る量が増えるほど、人が同じ見落としを毎回指摘するやり方は詰まります。レビューで見つけた失敗は、次回もっと早く見つけられる形へ戻します。

原因をコードで直す

使い方を間違えやすいAPIや、読まないと分からない境界は、説明を増やす前に構造を直す。

構造規約をLintで止める

Lintはコードの規約違反を機械的に見つける仕組みです。既存の検査を使い、固有の規約は必要に応じて自作し、変更時に自動実行します。

振る舞いをテストで固定する

値の条件や業務の動作など、実行結果で確かめるものはテストへ移す。

同じ注意を文章で読ませ続ける前に、コードの構造で防ぐか、Lint・テストで判定できないかを考えます。

機械で判定できることは、レビューより前に止める。



コンテキストには、判断の理由と作業の前提を残す

ここでいう**コンテキスト**は、依頼文、参照するコードや文書、実行結果など、AIが判断に使う情報です。文章の注意だけに頼らず、規則はLint・テストへ移し、操作できる範囲は権限で制限します。文書には、実装だけでは分からない理由や作業手順を残します。

リポジトリ全体で繰り返す前提

作業前に読むAGENTS.mdには、対象の範囲、ビルド手順、役割分担を短く残す。既存のコードから読み取れる説明を重ねすぎない。

作業固有の判断

目標、受け入れ基準、境界など、先ほど仕様として決めた作業固有の判断はDesign DocやPRの説明へ残す。

ソフトウェアの変更と一緒に文書も更新します。長い作業の引き継ぎでは、目的、確認済みの結果、未解決の点、次の確認を残します。古い指示を現行の方針と混ぜず、必要な情報へたどり着けるようにします。

理由と未決定の点を文書で渡し、守れる規則は実行環境でも守る。



機能を消す方が、作るより難しい

AIへ任せて作れる量が増えても、機能を消してよいかはコードだけでは決められません。追加では新しい動作と既存への影響を確かめます。削除ではさらに、**誰が今の動作を頼りにしているか**を調べます。その判断材料は、コードだけには残っていません。

利用実態と重要性を判断しにくい

短期間の利用記録だけでは、誰が、どんな状況で使っているかを捉えにくい。利用回数が少なくても、特定の顧客の業務や障害時の復旧に欠かせないことがあります。

依存が見えにくい

設定、データ、バッチ、帳票、運用手順、他チームの仕組みが、明示されないまま機能に依存していることがあります。

利用回数だけでも、コードだけでも、消してよいとは判断できない。



止める前に、利用者と依存先を確かめる

同じ機能について、利用者にとっての重要性和、仕組みが受ける影響を別々に確かめます。どちらか一方だけでは、安全に止められません。

利用者にとっての重要性

営業やCSには顧客の業務への影響を、デザイナーには利用する場面を確かめます。利用回数が少なくても、特定の業務や復旧時に欠かせない場合があります。

仕組みが受ける影響

エンジニアにはコードとデータの依存を、SREや運用担当には障害時の使われ方を確かめます。設定や帳票など、コードの外に依存が残ることもあります。

影響範囲を確かめる



関係者へ知らせ、新規利用を止める



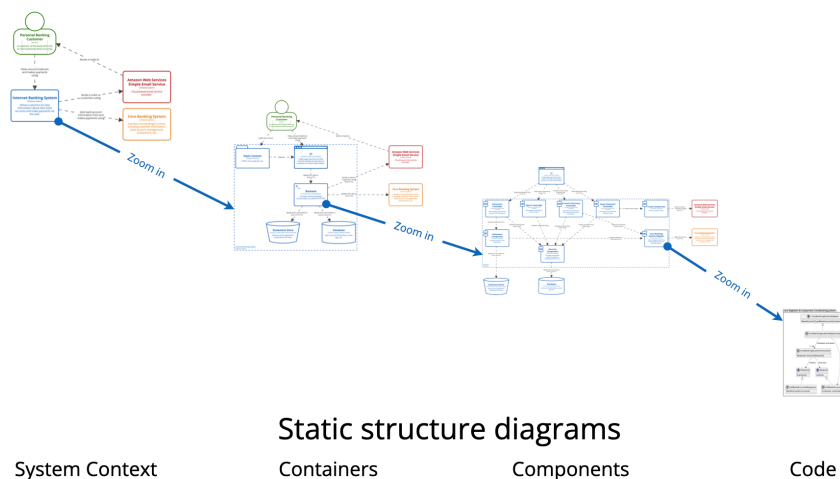
元に戻せる状態で停止する

利用者への重要性和構造上の影響を確かめ、関係者へ知らせてから段階的に止める。



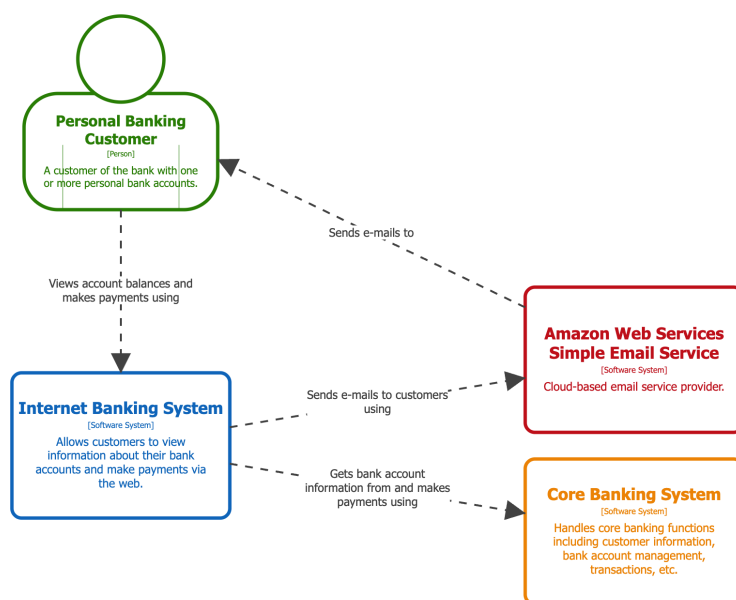
C4モデルは、構造の詳しさを四段階にそろえる

機能を消した影響を確かめるとき、まず全体を見て、必要な箇所だけ詳しくします。**C4モデル**は、システムと周囲の関係、内部のアプリとデータ、アプリ内の役割のまとめ、具体的なコードという四段階で構造を表します。それぞれをシステムコンテキスト、コンテナ、コンポーネント、コードと呼びます。



四段階すべてを描く必要はありません。この発表では、上の二つを中心に、「誰に影響するか」「どのアプリやデータに影響するか」を確かめます。

システムコンテキスト図で、利用者と外部連携を見る



System Context View: Internet Banking System
The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

対象システムと周囲の関係を見るのが、**システムコンテキスト図**です。対象システムを一つの箱として扱い、その周りに利用者と外部システムを置きます。

最初に確かめること

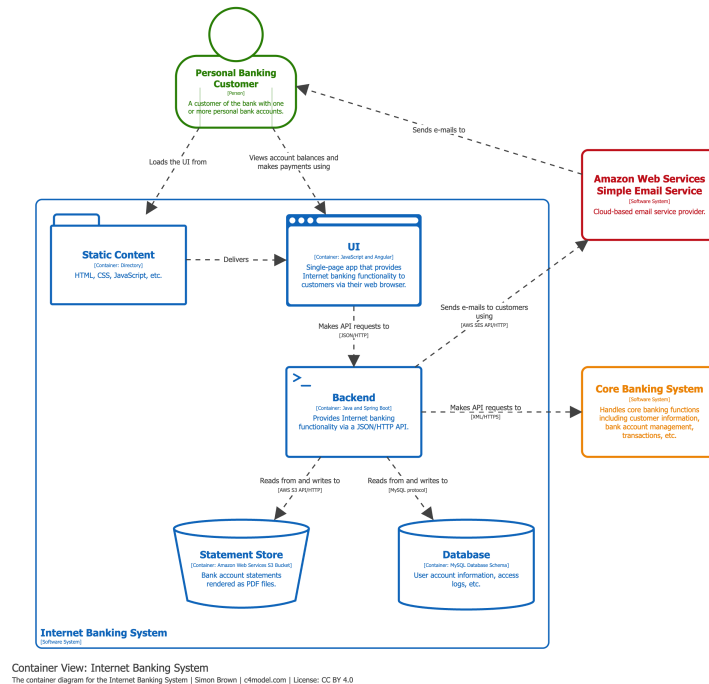
誰がその機能を使い、どのジョブを片付けているのか。どの外部システムから呼ばれるのか。止めたとき、誰の仕事や外部連携に影響するのか。

営業やCSが知る利用場面と、エンジニアが知る外部連携を、同じシステムの周りへ置けます。ここで、ジョブの理解と仕組みの範囲がつながります。

画像：Simon Brown, System Context diagram, CC BY 4.0

コンテナ図で、アプリ・データ・通信を見る

対象システムの中へ一段詳しく入るのが、**コンテナ図**です。アプリケーション、データストア、それらの通信に分けて表します。



C4モデルでいう「コンテナ」

Dockerコンテナに限りません。Webアプリ、API、バッチ、データベースなど、実行するアプリケーションかデータストアを指します。

次に確かめること

どのアプリ、データ、通信が依存しているか。止めたときに使える代替経路があるか。

システムコンテキスト図で影響を受ける相手を絞り、コンテナ図で変更や停止が波及する経路を絞ります。コンポーネントやコードを見るのは、経路をまだ特定できない箇所だけです。



検索の構造を、変更と相談の範囲へつなげる

銀行システムの図で見た読み方を、検索の架空例へ戻して使います。対象は商品検索システムです。検索APIの内部を変えたいとき、接続先と、影響を受ける利用者確かめます。

商品検索システム：コンテナ図の抜粋



矢印は依頼する向きで、応答は省略しています。画面が使う結果の形式やエラーを保てるか、検索用データを誰が更新するかを確認します。この抜粋では利用者・商品管理・運用の経路を省いているため、削除前にはそれらも調べます。

図から「変える範囲」「保つ接点」「確かめる相手」を取り出す。
描かれていない関係を、存在しない関係とは扱わない。

同じ箱と矢印を、同じ意味で読めるようにする

検索の図を共有しても、「検索API」を独立して動くアプリと読む人と、アプリ内の関数と読む人では、変更の見積もりが変わります。見た目をそろえる前に、何を表しているかをそろえます。

箱には、種類と役割を書く

アプリなのか、データストアなのか、アプリ内の処理のまとめりなのか。「検索条件を受け付ける」のように役割も添える。

矢印には、関係の内容を書く

単に「連携」ではなく、「検索条件を送る」「商品更新を通知する」と書く。依頼の向きなのか、データが流れる向きなのかも示す。

システムの境界と、チームの担当範囲は必ずしも一致しません。複数チームが変更するなら、データの意味や外部への応答を誰が判断するかも確認します。図の枠だけで担当者を推測しません。

図の目的は、同じ絵を見せることではなく、違う解釈に気づけるようにすること。



構造だけで足りないときは、一回の動作を追う

コンテナ図は何がつながるかを表します。順番や失敗時の扱いを確かめたいときは、**動的図**を使います。一つの操作について、どの要素がどの順番でやり取りするかを表す図です。

1. 検索画面 → 検索API 指定した納期で検索を依頼する。

2. 検索API → 検索用データ 条件に合う商品を問い合わせる。

3. 検索用データ → 検索API → 検索画面 結果を返し、画面へ表示する。

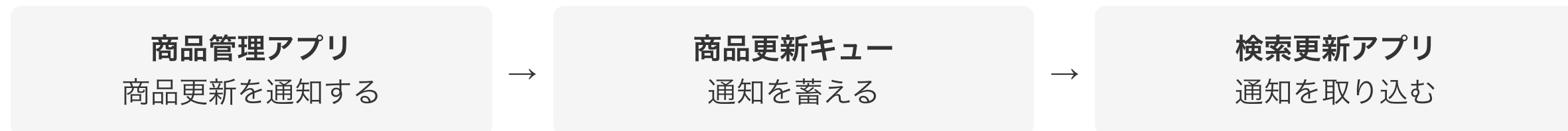
この順序を動的図にすると、2で応答がない場合に、どこで待つのをやめるか、画面へ何を返すかを話せます。実際の時間はログや計測で確認します。サーバーの配置や切り替え方を知りたいなら、稼働環境を表す**配置図**を別に使います。

構造のつながり、処理の順番、実際の待ち時間を分ける。
図を増やすのは、いまの図では判断に必要なことが分からないとき。



応答を待たない連携にも、依存は残る

先ほどの検索例では、呼び出した相手の応答を待ちました。一方、商品情報の更新は、通知を蓄えておき、検索側が後から取り込む方法もあります。こうした**非同期の連携**でも、変更の相談が不要になるわけではありません。



架空例の矢印は、通知が流れる向きです。通知の項目や意味を変えると、受け取る側も変更が必要になる場合があります。止める前には、受信者、処理待ちの通知、失敗時の再処理、データ形式を変更する担当を確かめます。

応答を待たないことと、相手に影響せず変えられることは違う。
「連携基盤」の一箱で済ませず、何を誰へ渡すかを見る。

検索の速さと、データ更新の反映は別に見る

商品更新を後から取り込む検索では、結果がすぐ返っても、変更前の納期が表示されることがあります。**応答時間**と、更新が検索へ届くまでの**反映時間**は別です。どちらも、ユーザーが判断に使える情報かどうかに関わります。

読むためのデータを用意する

商品管理のデータから、検索に必要な項目をまとめたデータを作る。検索は速くしやすくなるが、更新を取り込み続ける必要がある。

どこで何を確認するかを決める

検索時に許せる情報の古さと、注文確定時に確認する在庫・納期を分ける。反映が遅れた場合の表示と対応も決める。

許せる遅れは、エンジニアだけでは決められません。営業・CSが知る顧客の業務、デザイナーが考える表示、PMが判断する優先順位と合わせます。実際の反映時間と、古い情報で困った利用を確認します。

速く返すだけでなく、その情報でユーザーが判断してよいかを確認める。



キャッシュには、更新と復旧の設計も要る

キャッシュは、取得や計算の結果を保存して再利用する仕組みです。毎回同じ処理をせずに済みますが、元のデータが変わると、保存した結果が古くなることがあります。検索結果を保存するなら、その扱いまで設計します。

いつ使わなくするか

保存した結果の有効期限、商品更新時に消す範囲、更新に失敗した場合の対応を決める。閲覧権限や顧客ごとの条件が違う結果を混ぜない。

使えないときにどうするか

保存した結果がないときや、キャッシュが停止したときの動作を決める。すべての要求を元のDBへ流しても、処理しきれるとは限らない。

利用できた割合だけでなく、応答時間、情報の古さ、DB負荷、運用の手間を見ます。今の問い合わせや索引の改善で足りるなら、保存先を増やさない選択もあります。

減らせる処理と、増える更新・復旧の仕事を一緒に比べる。



図で決めた境界を、コードと確認手順につなげる

読み取り用データやキャッシュを加えたら、取得と更新の経路も図へ反映します。図では検索APIを経由するはずなのに、画面からデータへ直接アクセスしていたら、図だけでは影響を判断できません。人にもAIにも、図と実装が食い違う状態を前提として渡さないようにします。

図から実装を確かめられるようにする

アプリや処理の名前と、対応するリポジトリ・ディレクトリをつなぐ。接続先はコードや設定でも確認する。現状と変更案は分けて示す。

守る境界は、検査や権限にも反映する

禁止する依存はLintや構造のテストで検出する。データへの直接アクセスは権限でも制限する。必要な例外は、理由と見直す条件を残す。

図は変更と一緒に更新します。細部まで手書きで複製せず、コードから得られる情報は必要なときに生成します。AIには画像だけでなく、対応するコードや構成情報も渡し、推測と確認済みの事実を分けさせます。

図は判断を助ける。境界を守るのは、実装・権限・検査と、それを見直す人。



C4図だけでは、「残す・消す」は決められない

C4の構造図で、仕組みと依存関係を確認めます。必要なら動的図や配置図を補います。それでも、ジョブの重要性、実際の待ち時間、設計を選んだ理由は、別の記録や対話から確かめる必要があります。

バリューストリームマッピング

ジョブの発見からアウトカムの確認まで、どこで作業し、どこで待つかを見る

ワードリーマップ

ジョブに必要な要素と成熟度を並べ、どこへ投資し、どう運用するかを考える

C4図

誰が使い、どの外部システム、アプリ、データが依存しているかを確認する

利用記録・Design Doc・必要ならADR

実際の利用と設計した理由を残す。ADRは、大きな設計判断を後から単独で参照したいときに使う

一つの図へ詰め込まず、知りたいことに合う図や記録を使い分ける。



調べた結果から、残す・縮小・置き換える・消すを選ぶ

C4図で構造上の影響を確かめたら、利用者にとっての重要性和組み合わせます。この二つを分けて見ると、すぐに消せるものと、先に代替手段や依存の整理が必要なものを区別できます。

重要性が高い × 影響が大きい

残すか、代替手段へ段階的に移す。停止は最後にする。

重要性が高い × 影響が小さい

機能は残す。複雑さを減らせるなら、実装だけを簡素にする。

重要性が低い × 影響が大きい

先に依存を外すか、代替手段へ置き換える。その後で消す。

重要性が低い × 影響が小さい

関係者へ知らせ、元に戻せる状態を保ちながら消す。

Design DocやADRには、選んだ理由と見直す条件を残します。図や文書は、ソフトウェアの修正と一緒に更新できる詳しさへ絞ります。過去の判断は履歴として残し、現在の方針と区別します。

利用が少ないだけでは消さない。重要性和構造上の影響をそろえて、止め方を選ぶ。



機能を残して、実装だけを置き換えることもできる

ユーザーのジョブに必要でも、今の実装を維持し続ける必要があるとは限りません。検索基盤を置き換えるなら、利用者と呼び出し元が頼る動作を、新旧のどちらでも確かめられるようにします。

保つ動作を先に決める

納期条件と閲覧権限が守られるか。更新が必要な時間内に反映されるか。応答がないとき、利用者へ説明できるか。

新旧を同じ条件で比べる

同じ入力で結果と応答時間を比べる。過去の不具合も再現する。差が出たら、改善なのか、必要な動作の欠落なのかを判断する。

内部の関数名が変わるだけで壊れるテストと、利用者が頼る動作を確かめるテストは役割が違います。後者を新旧へ共通に使い、実装を変えても必要な条件が守られるかを確認します。

生成し直せることと、安全に置き換えられることは別。
守る動作を、置き換えるコードの外でも確かめられるようにする。



データ移行は、コピーが終わっても終わらない

検索用データを新しい保存先へ移す間も、商品の更新は続きます。コピーした件数が一致しただけでは、その間の更新や削除まで反映されたとは分かりません。移行中にどちらへ書き、どちらから読むかを決めます。

更新を取りこぼさない

既存データを移し、その間の更新・削除も追いつかせる。処理が重複したり順番が変わったりしても、結果が壊れないかを試す。

読んだ結果を比べる

件数だけでなく、納期、閲覧権限、削除済みの商品、更新の反映を比べる。違いを説明できるまで、切り替える範囲を広げない。

戻した後も更新を失わない

旧環境へ戻すなら、切り替え後の更新も扱えるかを確認する。古いコピーへ戻すだけでは、新しい変更を失う場合がある。

新旧の両方へ書けば安全、とは限りません。片方だけ失敗した場合の検出と修復が必要です。書き込みを止められるなら、停止時間を合意して移す方が単純なこともあります。

データを移すだけでなく、移している間と、戻すときの動作を確かめる。



切り替えは、影響と戻せる範囲に合わせて進める

新しい実装がテストを通過しても、本番のすべての使われ方を確かめたわけではありません。画面の一部を試す変更と、全利用者が使う保存形式の変更では、必要な確認と観察期間が変わります。

限った範囲で試す

まず検証環境で確認し、本番は対象を絞る。検索結果の差、エラー、利用者のつまずきを見る。

戻す条件を決める

どんな異常で止め、誰が判断するかを決める。旧実装へ戻す手順を、切り替え前に確かめる。

データも戻せるかを見る

新実装が書いたデータを旧実装が読めるか確認する。コードを戻すだけで済まない変更は、移行方法を別に検証する。

本番で分かった例外は、その場の修正だけで終わらせず、テストと設計の理由へ戻します。次の人やAIが同じ箇所を変えても、必要な動作を失わないためです。

変更の小ささは、差分の行数では決まらない。
影響を受ける範囲と、失敗を見つけて戻す方法で決める。



置き換えた後は、古い経路を片付ける

新旧を切り替えられる状態は、移行中には役立ちます。ただし、古いAPI、設定、データ形式を残し続けると、次の変更でも両方を理解し、確認する必要があります。

廃止できる条件を確かめる

呼び出し元の移行、定期処理の利用、古いデータの扱いを確認する。戻すために残す期間と、その後の復旧方法も決める。

役目を終えたものを減らす

古い経路と設定を削除し、現行の図と手順を更新する。必要な動作のテストと、移行を決めた理由が残す。

過去の判断記録は、現在も従う指示とは分けて保存します。古い方針には変更先を示し、なぜ切り替えたかを後から追えるようにします。履歴まで消すと、次の変更で同じ理由を調べ直すことになります。

新しく動くだけで終わらせず、次の変更で読むもの・守るものを減らす。



いつやめるかは、作る前に決める

機能を消すのが難しいからこそ、削除の判断を後回しにしません。作る前に置いた前提も、利用や市場の変化で古くなります。半年計画の例では、4ヶ月目まで中止を判断できませんでした。やめる条件と判断日を、機能を作る前に書いておけば、継続か中止かをもっと早く決められます。「2ヶ月の調査が終わる7月16日に、続けるかを決める」のように。

ジョブが別の手段で片付いている
マネージドサービスや既存機能で足りる。置き換える

想定したジョブには使われていない
別のジョブを確かめるか、機能を消す

維持費や変更リスクが、得られる価値を上回る
代替手段と元に戻す方法を確認、縮小・置き換え・削除を選ぶ

判断日には、利用記録、問い合わせ、依存先、障害時の代替手段を見直します。やめる条件に当てはまれば、縮小・停止・置き換えを選びます。材料が足りなければ、追加で確かめることと次の判断日を決めます。使った費用の大きさだけを、続ける理由にはしません。

「この条件になったらやめる」と「いつ判断するか」を、作る前のDesign Docに書く。
小さな変更ならPRの説明に、大きな設計判断を後から単独で参照したいときだけADRにも残す。

まとめ 実装を速めても待ちは残る

AI支援で実装時間が短くなっても、ジョブの発見から結果の確認までには、実装以外の時間があります。増えたアウトプットを受け入れる仕事と、利用後の結果を確かめる仕事も一緒に見ます。

アウトプットを作る

設計・実装・テスト。AI支援で短くしやすく、PR数や生成量にも表れやすい。

待ち

優先順位の判断、レビュー、他の役割との合意。仕事の進め方を変えなければ残る。

アウトカムを確かめる

本番で使われ、狙った変化が起きたかを確かめる。リリースしただけでは分からない。

**30/70の例では、実作業を半分にしても全体は15%しか縮まらない。
実装の外に待ちが残り、アウトプットの増加だけではアウトカムの変化が分からない。**



まとめ 作る・任せる・やめるを決める

「作る・任せる・やめる」は、次の順序で考えます。

作る

最近困った場面と、片付けたいジョブ、確かめたいアウトカムを見る。既存の手段で足りるなら作らない。

任せる

前提を仕様へ書き、AIの計画を確認する。テストと計測で確かめられ、元に戻せる範囲まで任せる。

やめる

利用者への重要性和構造上の影響を調べる。必要な動作を保ち、置き換えや削除を選ぶ。条件と判断日は作る前に決める。

AIは「どう作るか」の候補を増やす。

役割ごとの前提を図と対話で確かめ、チームが「作る・任せる・やめる」を決める。



3>SHAKE

AIは作れる。

作るべきかを決めるのが私たちの仕事。

職能の壁を越えて、価値が届くまでの流れを設計する

ありがとうございました

@nwiizo